TITLE

HOUSE PRICE PREDICTION

A REPORT ON

PROJECT BASED LEARNING

IN

AI TOOLS,TECHNIQUES AND APPLICATIONS

M.Lohith Kumar-20331A1274

M.Bhanu Prakash-20331A1275

M.Vinitha Varshini-20331A1276

MVGR COLLEGE OF ENGINEERING (A)

INFORMATION TECHNOLOGY

LAB Incharge Dr B Anjana Devi HOD IT

PROBLEM STATEMENT :House Price Prediction

INTRODUCTION

Housing price trends are not only the concern of buyers and sellers, but it also indicates the current economic situation. There are many factors which has impact on house prices, such as numbers of bedrooms and bathrooms. Even the nearby location, a location with a great accessibility to highways, expressways, schools, shopping malls and local employment opportunities contributes to the rise in house price. Manual house predication becomes difficult, hence there are many systems developed for house price prediction. This system aim is to make a model which can give us a good house pricing prediction based on other variables. We are going to use Linear Regression for this dataset and hence it gives a good accuracy.

LINEAR REGRESSION:

Linear regression is a **linear model**, e.g. a model that assumes a linear relationship between the input variables (x) and the single output variable (y). More specifically, that y can be calculated from a linear combination of the input variables (x).
When there is a single input variable (x), the method is referred to as **simple linear regression**. When there are **multiple input variables**, literature from statistics often refers to the method as multiple linear regression.
Different techniques can be used to prepare or train the linear regression equation from data, the most common of which is called **Ordinary Least Squares**.

HARDWARE SPECIFICATIONS

SYSTEM PROCESSOR: Intel core i5
RAM : 8GB

SOFTWARE SPECIFICATIONS

OPERATING SYSTEM : Windows 10
LANGUAGE : Python

EXISTING SYSTEM:

The existing system involves **calculation of** house prices without the necessary prediction about future market trends and price increase. ... By

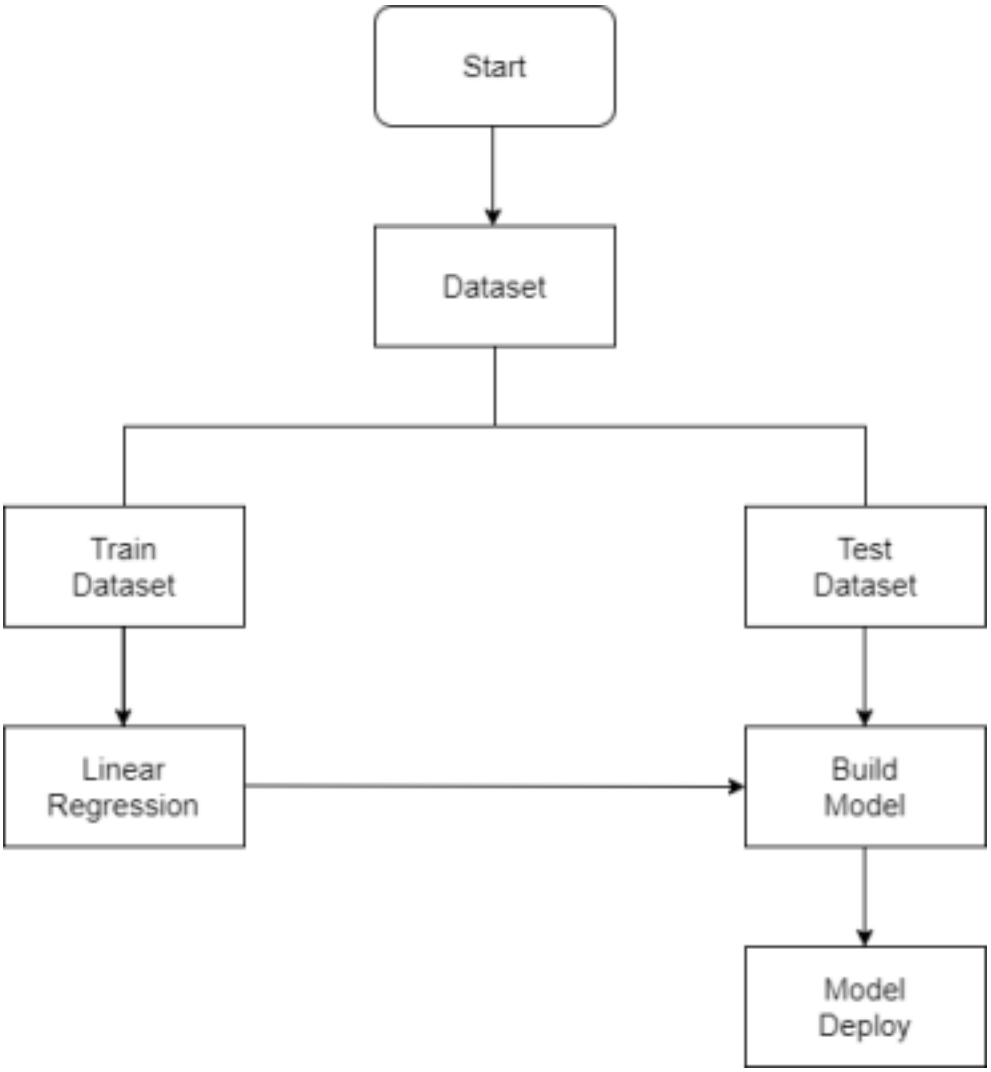
analyzing previous market trends and price ranges, and also upcoming developments future prices will be predicted.

PROPOSED SYSTEM:

A Machine Learning Algorithm is developed using Linear Regression where train set is considered and the values of a test set are predicted.

The manual method which is currently used in the market is out dated and has high risk. So as to overcome this fault, there is a need for an updated and automated system. Data mining algorithms can be used to help investors to invest in an appropriate estate according to their mentioned requirements. Also the new system will be cost and time efficient. This will have simple operations. The proposed system works on Linear Regression Algorithm.

SYSTEM ARCHITECTURE:



ALGORITHM

IMPLEMENTATION

MODULES USED

NUMPY:

NumPy is a Python library used for working with arrays.

It also has functions for working in domain of linear algebra, fourier transform, and matrices.
NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.
NumPy stands for Numerical Python.
In Python we have lists that serve the purpose of arrays, but they are slow to process.
NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy.
Arrays are very frequently used in data science, where speed and resources are very important.

PANDAS:

Pandas is a Python library used for working with data sets.

It has functions for analyzing, cleaning, exploring, and manipulating data.

The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" .

Pandas allows us to analyze big data and make conclusions based on statistical theories.

Pandas can clean messy data sets, and make them readable and relevant.

Regression Evaluation Metrics

Here are three common evaluation metrics for regression problems:

Mean Absolute Error (MAE) is the mean of the absolute value of the errors:

$$\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Mean Squared Error (MSE) is the mean of the squared errors:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) is the square root of the mean of the squared errors:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

SOURCE CODE:

```
import pandas as pd
import numpy as np
import seaborn as sns
from google.colab import drive
drive.mount('/content/sdrive',force_remount=True)
HouseDF = pd.read_csv('/content/sdrive/MyDrive/data.csv')

HouseDF.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4600 entries, 0 to 4599
Data columns (total 18 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   date           4600 non-null   object
 1   price          4600 non-null   float64
 2   bedrooms       4600 non-null   float64
 3   bathrooms      4600 non-null   float64
 4   sqft_living    4600 non-null   int64
 5   sqft_lot       4600 non-null   int64
 6   floors         4600 non-null   float64
 7   waterfront     4600 non-null   int64
 8   view           4600 non-null   int64
 9   condition      4600 non-null   int64
 10  sqft_above     4600 non-null   int64
 11  sqft_basement  4600 non-null   int64
 12  yr_built       4600 non-null   int64
 13  yr_renovated   4600 non-null   int64
 14  street         4600 non-null   object
 15  city           4600 non-null   object
 16  statezip       4600 non-null   object
 17  country        4600 non-null   object
dtypes: float64(4), int64(9), object(5)
memory usage: 647.0+ KB
```

HouseDF.describe()

Output:

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	sqft_above	sqft_base
count	4.600000e+03	4600.000000	4600.000000	4600.000000	4.600000e+03	4600.000000	4600.000000	4600.000000	4600.000000	4600.000000	4600.00
mean	5.519630e+05	3.400870	2.160815	2139.346957	1.485252e+04	1.512065	0.007174	0.240652	3.451739	1827.265435	312.08
std	5.638347e+05	0.908848	0.783781	963.206916	3.588444e+04	0.538288	0.084404	0.778405	0.677230	862.168977	464.13
min	0.000000e+00	0.000000	0.000000	370.000000	6.380000e+02	1.000000	0.000000	0.000000	1.000000	370.000000	0.00
25%	3.228750e+05	3.000000	1.750000	1460.000000	5.000750e+03	1.000000	0.000000	0.000000	3.000000	1190.000000	0.00
50%	4.609435e+05	3.000000	2.250000	1980.000000	7.683000e+03	1.500000	0.000000	0.000000	3.000000	1590.000000	0.00
75%	6.549625e+05	4.000000	2.500000	2620.000000	1.100125e+04	2.000000	0.000000	0.000000	4.000000	2300.000000	610.00
max	2.659000e+07	9.000000	8.000000	13540.000000	1.074218e+06	3.500000	1.000000	4.000000	5.000000	9410.000000	4820.00

HouseDF.columns

Output:

```
[ ] HouseDF.columns

Index(['date', 'price', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
       'floors', 'waterfront', 'view', 'condition', 'sqft_above',
       'sqft_basement', 'yr_built', 'yr_renovated', 'street', 'city',
       'statezip', 'country'],
      dtype='object')
```

#splitting our data into x nd y lists..x=features to train, y=target variable

```
X = HouseDF[['date', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
'floors', 'waterfront', 'view', 'condition', 'sqft_above',
'sqft_basement', 'yr_built', 'yr_renovated', 'street', 'city',
'statezip','country']]
y = HouseDF['price']
```

#splitting into train and test

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4,
random_state=101)
```

#creating and training the lgmodel

```
from sklearn.linear_model import LinearRegression
lm = LinearRegression()
X = X_train.apply(pd.to_numeric, errors='coerce')
y = y_train.apply(pd.to_numeric, errors='coerce')
X.fillna(0, inplace=True)
y.fillna(0, inplace=True)

lm.fit(X,y)
```

Output:

```
LinearRegression()
```

```
print(lm.intercept_)
```

Output:

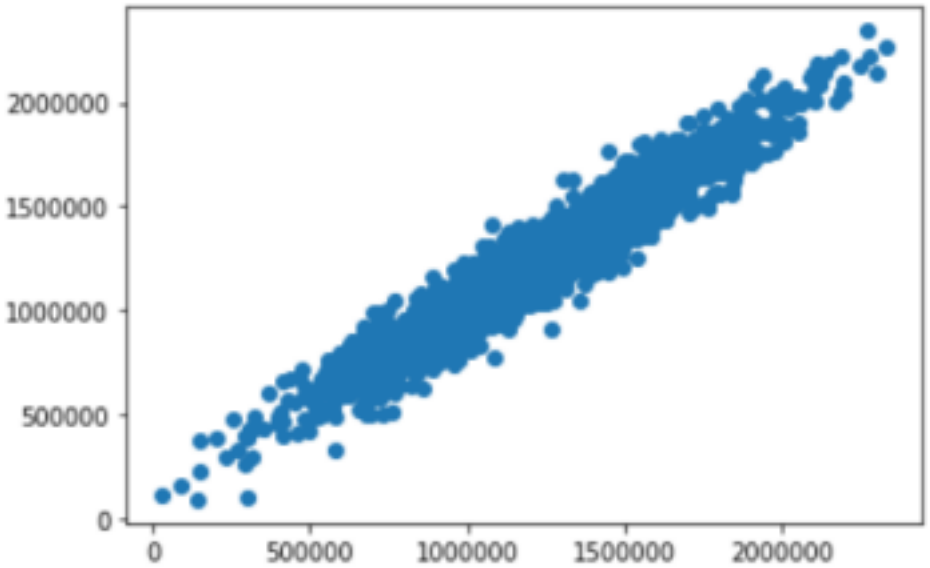
```
[7] print(lm.intercept_)

    5399305.140280319
```

```
coeff_df =
pd.DataFrame(lm.coef_,X.columns,columns=['Coefficient']) coeff_df
```

Output:

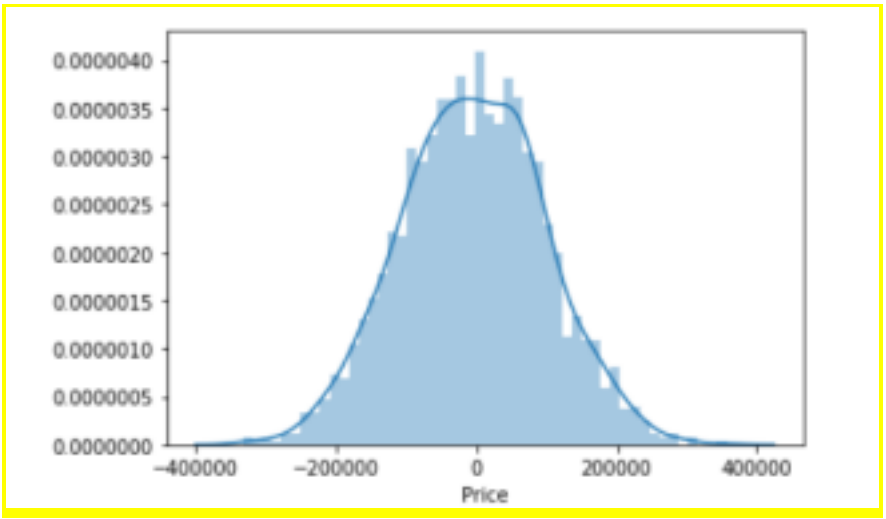
```
import matplotlib.pyplot as plt

predictions = lm.predict(X_test)

plt.scatter(y_test,predictions)
```

Output:




```
sns.distplot((y_test-predictions),bins=50);
```

Output:



CONCLUSION:

In today's real estate world, it has become tough to store such huge data and extract them for one's own requirement. Also, the extracted data should be useful. The system makes optimal use of the Linear Regression Algorithm. The system makes use of such data in the most efficient way. The linear regression algorithm helps to fulfil customers by increasing the accuracy of estate choice and reducing the risk of investing in an estate. A lot's of features that could be added to make the system more widely acceptable. One of the major future scopes is adding estate database of

more cities which will provide the user to explore more estates and reach an accurate decision. More factors like recession that affect the house prices shall be added. In-depth details of every property will be added to provide ample details of a desired estate. This will help the system to run on a larger level.

INFERENCE: Pandas **describe()** is used to view some basic statistical details like percentile, mean, std etc. of a data frame or a series of numeric values. coeffdf says: Holding all other features fixed, a 1 unit increase in Avg. Area House Age is associated with an increase of $164883.28 .Holding all other features fixed, a 1 unit increase in Avg. Area Number of Bedrooms is associated with an increase of $2233.80 .

REFERENCES:

https://towardsdatascience.com/predicting-house-prices-with-linear-regression-machine-learning-from-scratch-part-ii-47a0238aeac1